

Love of Variety Model and Simulation

Tate Mason

April 6, 2026

1 Model

1.1 Consumer's Problem

The consumer's problem is to maximize utility subject to their choice set. The utility function is as follows:

$$U_{ijt} = \beta_i X_{jt} - \gamma_i (\Sigma_{ijt})^2 + \kappa a_{jt} - \alpha_j p_{jt} + \xi_j + \epsilon_{ijt} \quad (1)$$

$$\Sigma_j = \bar{X}_{it} - X_{jt}$$

Where U_{ijt} is the utility of consumer i from product j at time t . X_{jt} is the attribute of product j at time t . γ_i denotes a cost of sameness. Σ_j is the sum of product attributes chosen until time t . Note, γ_i has a standard normal distribution. ξ_j captures utility from unobserved quality. Finally, ϵ_{ijt} is a random error term distributed T1EV.

$$P_{ijt} = \frac{e^{U_{ijt}}}{1 + \sum_k e^{U_{ikt}}} \quad (2)$$

Where the denominator sums over all products k in the choice set at time t , with the outside option.

1.2 Firm's Problem

The firm will maximize profits by choosing how to advertise to the consumer according to their perceptions surrounding the distribution of γ .

$$\pi_{jt} = s_{jt}(p_{jt}, a_{jt})(p_{jt} - mc_{jt}) - \kappa a_{jt} * * 2 \quad (3)$$

The firm's profits are a function of the market share s_{jt} , which in this case is the probability our individual consumer chooses product j at time t . This is a function of the price p_{jt} and the advertising level a_{jt} . The share is scaling the difference in price and marginal cost. The firm also has to pay for advertising, which is quadratic in the level of advertising.

For now, we will assume that $a_{jt} \sim \text{lognormal}(\frac{(\bar{X}_{it} - X_{jt})^2}{1 + \sum_k (X_{it} - X_{kt})^2}, \sigma_a)$. This means that the firm is more likely to advertise when their product is more different from the average product in the market. The variance σ_a captures the uncertainty in the firm's advertising decision. Later, I would like to make the advertising decision endogenous to the firm's beliefs about the distribution of γ in the consumer population.

$$V(a_{jt} | \lambda_{jt}(\gamma_i)) = \max_{\pi_{jt}} \{ \pi_{jt} + \int_{\gamma} V'(a_{j,t+1} | \lambda_{j,t+1}(\gamma_i)) d\lambda(\gamma) \} \quad (4)$$

Here, $\lambda(\gamma)$ is the firm's belief about the distribution of γ in the consumer population. Σ_j is

the sum of product attributes chosen until time t . a_{t-1} is the advertising level at time $t - 1$. The firm chooses a to maximize profits based on its current beliefs and the expected future value of advertising. Note, the distribution of $\lambda(\gamma) \sim N(\frac{1}{\Sigma_j}, \sigma_\lambda)$. This states they use past purchases to update their mean belief about the distribution of γ , but they have some variance in that belief.

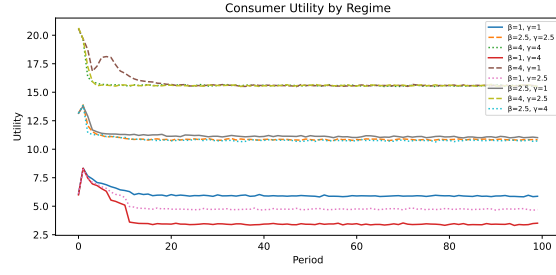
2 Results

Parameter	Value	Description
β_i	{0.2, 0.5, 0.8}	Consumer preference for product attributes
γ_i	{0.2, 0.5, 0.8}	Consumer cost of sameness
κ	2	Advertising cost parameter
α_j	0	Price sensitivity
p_{jt}	2	Price of product j at time t
mc_{jt}	1	Marginal cost of product j at time t
β_{DF}	0.9	Discount factor for the firm

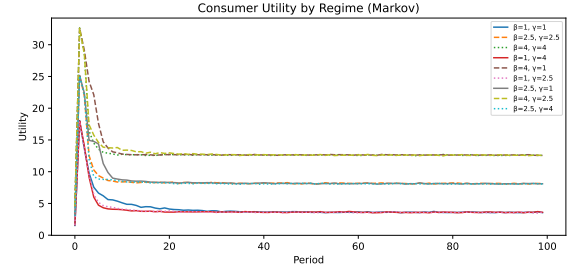
Table 1: Simulation Parameters

Consumer utility decreases over time in both specifications. This is because the consumer will choose similar products over time, which in turn hurts their utility via γ . The firm will choose a fixed advertising level, equally advertising to each product.

Choice probabilities are the truly interesting part. In the fixed specification, we see probabilities bunched around 0.2, which is the level of advertising. Interestingly, we also see some substitution towards either the outside option or other products. For instance, no one chooses product 1 after around $t = 85$. When looking at the Markov specification, we see a lot of variation. Again, looking at product 1, we see that probability of choice does not go above 0.15 until $t = 11$. We also see that people fully stop choosing the product after around $t = 40$. This is because advertising is based on inference of γ_i , based on the chosen products.

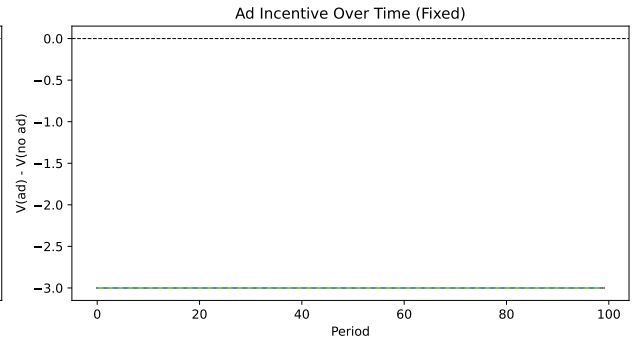
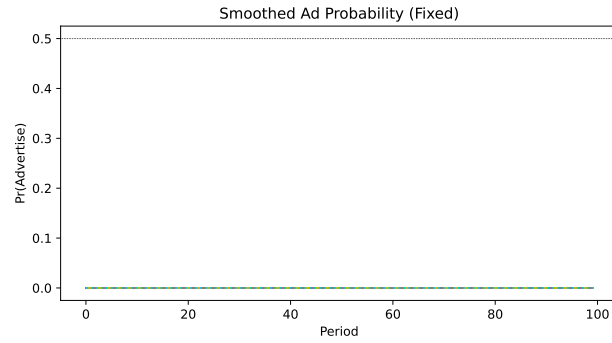


(a) Consumer Utility by Regime (Fixed Advertising $a = 0.2$)

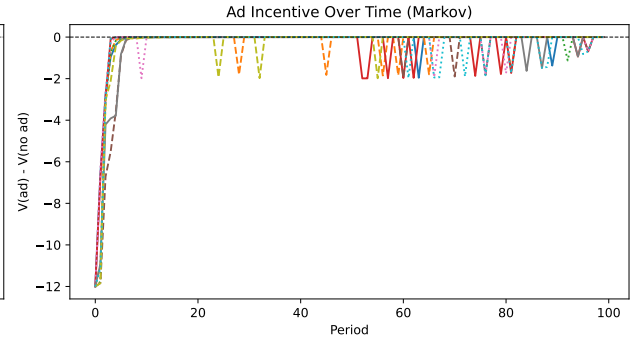
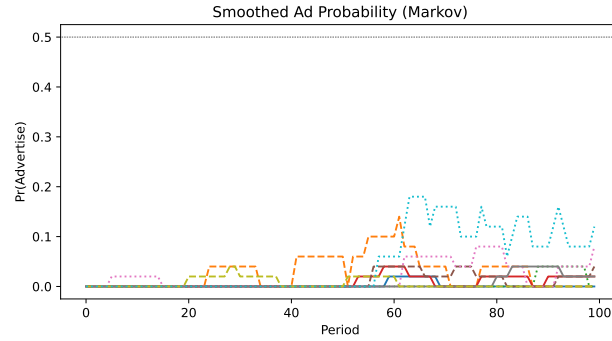


(b) Consumer Utility by Regime (Markov Advertising)

Figure 1: Consumer Utility across Regimes

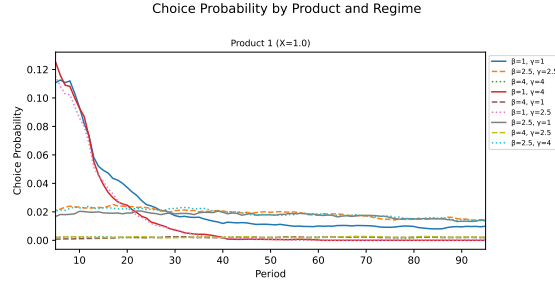


(a) Firm Advertising by Regime (Fixed Advertising $a = 0.2$)

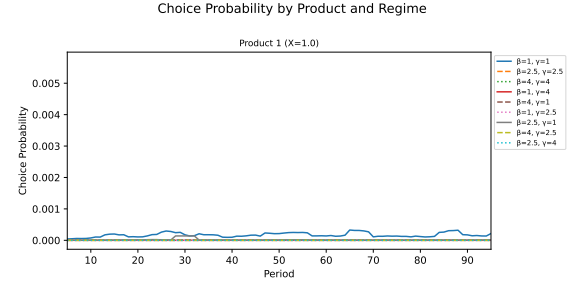


(b) Firm Advertising by Regime (Markov Advertising)

Figure 2: Firm Advertising across Regimes

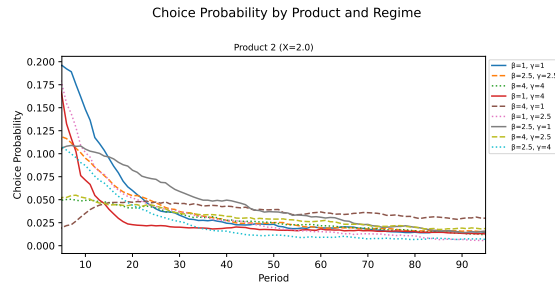


(a) Choice Probability of Product 1 by Regime
(Fixed Advertising $a = 0.2$)

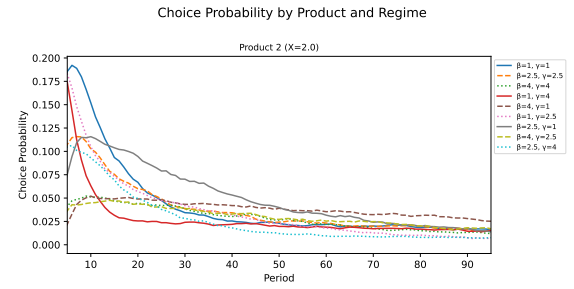


(b) Choice Probability of Product 1 by Regime
(Markov Advertising)

Figure 3: Choice Probability of Product 1 across Regimes

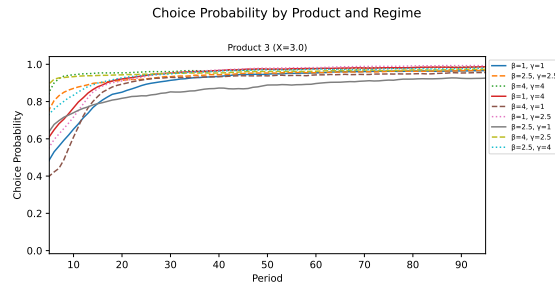


(a) Choice Probability of Product 2 by Regime
(Fixed Advertising $a = 0.2$)

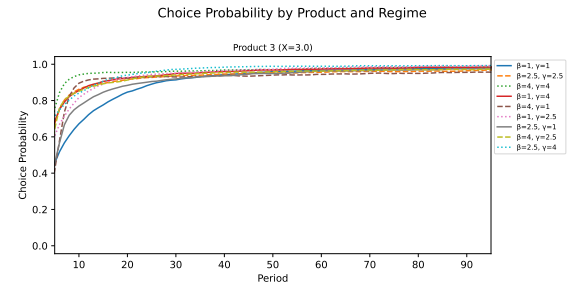


(b) Choice Probability of Product 2 by Regime
(Markov Advertising)

Figure 4: Choice Probability of Product 2 across Regimes

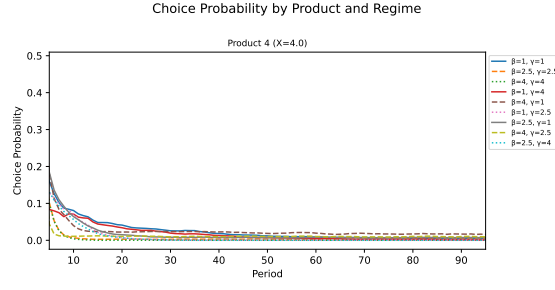


(a) Choice Probability of Product 3 by Regime
(Fixed Advertising $a = 0.2$)

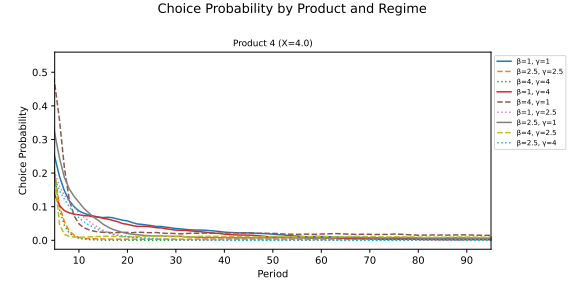


(b) Choice Probability of Product 3 by Regime
(Markov Advertising)

Figure 5: Choice Probability of Product 3 across Regimes

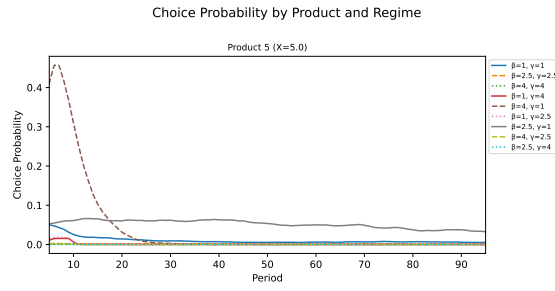


(a) Choice Probability of Product 4 by Regime
(Fixed Advertising $a = 0.2$)

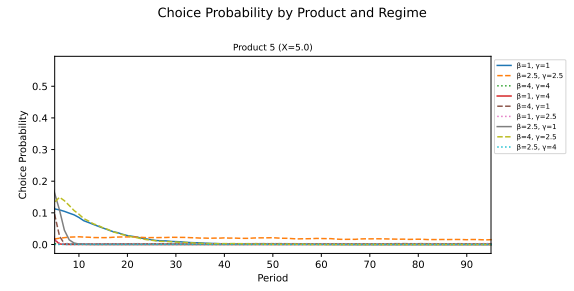


(b) Choice Probability of Product 4 by Regime
(Markov Advertising)

Figure 6: Choice Probability of Product 4 across Regimes



(a) Choice Probability of Product 5 by Regime
(Fixed Advertising $a = 0.2$)



(b) Choice Probability of Product 5 by Regime
(Markov Advertising)

Figure 7: Choice Probability of Product 5 across Regimes

3 Code

```
import pandas as pd
import os
import numpy as np
import scipy as sp
from scipy.special import logsumexp
import matplotlib.pyplot as plt
import seaborn as sns
from tabulate import tabulate
from collections import namedtuple
from progress.bar import IncrementalBar

"""
This file will be used to do initial simulation of the love of variety problem:
----Consumer Problem:  $U_{ijt} = \beta_i X_{jt}^{-\gamma_i} (\Sigma_{jt})^{\alpha_{ijt} + \epsilon_{ijt}}$ 
-----s.t.  $\gamma_i \sim N(0, \sigma_{\gamma})$ ;  $\Sigma_{jt} = \sum_t T_{jt} X_{jt}$ 
----Firm Problem:  $V(a_t; \Sigma_{jt}, \lambda_t(\gamma_i)) = \{ \pi_t + \int \gamma_i V(a'; \Sigma_{jt} + 1, \lambda_t + 1(\gamma_i)) d\lambda_t(\gamma_i) \}$ 
-----s.t.  $\pi_t = (1 + a) p_t - c$ 
----- $\lambda_t(\gamma_i) \sim N(1/\Sigma_{jt}, \sigma_{\lambda})$ ,  $a_t \in \{0, 1\}$ 
Structure:
----1) solve consumer's choice problem
----2) compute individual shares of products
----3) compute firm's problem, choosing optimal  $a$ 
----4) find partial equilibrium
Landscape:
----1)  $T = 100$ 
----2)  $J = 5$ 
----3)  $i = 1$ 
----4)  $\gamma = \text{range}(0.2(0.3)0.8)$ ,  $\beta = \text{range}(0.2(0.3)0.8)$ 
Functions:

Libraries Used:
----numpy: used for numerical operation
----scipy: used for optimization routines
----matplotlib, seaborn: used for plotting
----tabulate: better output text results

"""

#=====#
# Setup and Parameter Definition                                     #
#=====#

output_dir = os.chdir('../Output/')

rng = np.random.default_rng(seed=219)
```

```

# Parameterization
T = 100
sigma_gamma = [0.5, 1.0, 1.5]
kappa = 3

regimes = [
    (1, 1, " =1,  $\omega$  =1", '-'),
    (2.5, 2.5, " =2.5,  $\omega$  =2.5", '--'),
    (4, 4, " =4,  $\omega$  =4", ':'),
    (1, 4, " =1,  $\omega$  =4", '-'),
    (4, 1, " =4,  $\omega$  =1", '--'),
    (1, 2.5, " =1,  $\omega$  =2.5", ':'),
    (2.5, 1, " =2.5,  $\omega$  =1", '-'),
    (4, 2.5, " =4,  $\omega$  =2.5", '--'),
    (2.5, 4, " =2.5,  $\omega$  =4", ':')
]

prod_space = [1.0, 2.0, 3.0, 4.0, 5.0]
J = len(prod_space)
S = 1000
sigma_x = rng.uniform(0.5, 1.5)
X_bar = np.mean(prod_space)

ConsResult = namedtuple('ConsResult', ['x_chosen_all', 'x_bar_all', 'X_jt_all', 'V_all',
    'prob_all', 'U_all', 'll'])

# Consumer Choice Problem

def simulate_cons(beta, gamma, kappa, sigma_gamma, T, J, X_bar, sigma_x, rng, S=1000):
    x_chosen_all = np.zeros((S, T))
    x_bar_all = np.zeros((S, T))
    X_jt_all = np.zeros((S, T, J))
    V_all = np.zeros((S, T, J))
    ll_all = np.zeros(S)
    prob_all = np.zeros((S, T, J))
    U_all = np.zeros((S, T))

    for s in range(S):

        epsilon_ijt = rng.gumbel(0,1, size=(T,J))

        x_chosen = np.zeros(T) # empty vector of choices per period
        X_jt = np.zeros((T,J))
        x_bar = np.mean(prod_space)
        #x_bar[0] = np.mean(prod_space)

        Sigma = np.zeros((T,J)) # empty vector for the sum of characteristics

```



```

V = np.zeros((T, J))
chosen_idx = np.zeros(T, dtype=int)

X_jt[0] = np.array(prod_space)
a = np.zeros(T)

a[0] = 0
u0 = beta*X_jt[0] - gamma*0 + a[0] + epsilon_ijt[0]
V[0] = u0
chosen_idx[0] = np.argmax(u0)
x_chosen[0] = X_jt[0, chosen_idx[0]]
Sigma[0, chosen_idx[0]] = x_chosen[0]

for t in range(1, T):
    u = np.zeros(J)
    a[t] = 1.0
    #x_bar[t] = np.mean(x_chosen[:t])
    X_jt[t] = np.array(prod_space)
    Sigma[t] = Sigma[t-1].copy()
    Sigma[t, chosen_idx[t-1]] += (x_bar - x_chosen[t-1])
    u = beta*X_jt[t] - gamma*(Sigma[t])**2 + kappa*a[t] + epsilon_ijt[t] #
        love variety
    V[t] = u
    chosen_idx[t] = np.argmax(V[t])
    x_chosen[t] = X_jt[t, chosen_idx[t]]

ll = -np.sum(V[np.arange(T), chosen_idx]-logsumexp(V,axis=1,keepdims=True))
prob = np.exp(V) / np.exp(V).sum(axis=1, keepdims=True)
prob_all[s] = prob
x_chosen_all[s] = x_chosen
x_bar_all[s] = x_bar
X_jt_all[s] = X_jt
V_all[s] = V
U_all[s] = V[np.arange(T), chosen_idx]
ll_all[s] = ll
return ConsResult(
    x_chosen_all.mean(axis=0), # shape (T,)
    x_bar_all.mean(axis=0),    # shape (T,)
    X_jt_all.mean(axis=0),    # shape (T, J)
    V_all.mean(axis=0),       # shape (T, J)
    prob_all.mean(axis=0),    # shape (T,J)
    U_all.mean(axis=0),       # shape (T,)
    ll_all.mean())            # scalar

colors = plt.cm.tab10(np.linspace(0,1,len(regimes)))

fig, ax = plt.subplots(figsize=(8, 4))

```

```

for (beta, gamma, label, ls), color in zip(regimes, colors):
    x_chosen, x_bar, X_jt, V, prob, U, ll = simulate_cons(
        beta, gamma, kappa, sigma_gamma[1], T, J, X_bar, sigma_x, rng
    )
    ax.plot(np.arange(T), U, label=label, color=color, linewidth=1.5, linestyle=ls)

ax.set_xlabel("Period")
ax.set_ylabel("Utility")
ax.set_title("Consumer_Utility_by_Regime")
ax.legend(fontsize=7, loc="upper_right", bbox_to_anchor=(1,1))
plt.tight_layout()
plt.savefig("consumer_utility_by_regime.pdf", bbox_inches='tight', format='pdf')
plt.close()

```

Consumer Choice by Regime

```

for j in range(J):
    fig, ax = plt.subplots(figsize=(8, 4))
    for (beta, gamma, label, ls), color in zip(regimes, colors):
        x_chosen, x_bar, X_jt, V, prob, U, ll = simulate_cons(
            beta, gamma, kappa, sigma_gamma[1], T, J, X_bar, sigma_x, rng
        )
        prob_smooth = np.convolve(prob[:, j], np.ones(5)/5, mode='same')
        ax.plot(np.arange(T), prob_smooth, label=label, color=color, linewidth=1.5,
            linestyle=ls)

    ax.set_title(f"Product_{j+1}_X={prod_space[j]:.1f})", fontsize=9)
    ax.set_xlabel("Period")
    ax.set_xlim(5, 95)
    ax.set_ylabel("Choice_Probability")
    ax.legend(fontsize=7, loc="upper_left", bbox_to_anchor=(1, 1))
    fig.suptitle("Choice_Probability_by_Product_and_Regime", fontsize=13, y=1.01)
    plt.tight_layout()
    plt.savefig(f"choice_prob_product_{j+1}.pdf", bbox_inches='tight', format='pdf')
    plt.close()

```

```

def simulate_cons_markov(beta, gamma, kappa, sigma_gamma, T, J, X_bar, sigma_x, rng, S
=1000):
    x_chosen_all = np.zeros((S, T))
    x_bar_all = np.zeros((S, T))
    X_jt_all = np.zeros((S, T, J))
    V_all = np.zeros((S, T, J))
    ll_all = np.zeros(S)
    prob_all = np.zeros((S, T, J))
    U_all = np.zeros((S, T))

    for s in range(S):

```

```

epsilon_ijt = rng.gumbel(0,1, size=(T,J))
eps_markov = rng.standard_normal(size=T)

x_chosen = np.zeros(T) # empty vector of choices per period
X_jt = np.zeros((T,J))
x_bar = np.mean(prod_space)
#x_bar[0] = np.mean(prod_space)

Sigma = np.zeros((T,J)) # empty vector for the sum of characteristics

V = np.zeros((T, J))
chosen_idx = np.zeros(T, dtype=int)

X_jt[0] = np.array(prod_space)
a = np.zeros(T)

a[0] = 0
u0 = beta*X_jt[0] - gamma*0 + a[0] + epsilon_ijt[0]
V[0] = u0
x_chosen[0] = X_jt[0, chosen_idx[0]]
Sigma[0, chosen_idx[0]] = x_chosen[0]

for t in range(1, T):
    u = np.zeros(J)
    X_jt[t] = np.array(prod_space)
    a[t] = kappa*(x_bar - x_chosen[t-1])**2 # Markovian promotion based on
        distance from mean
    Sigma[t] = Sigma[t-1].copy()
    Sigma[t, chosen_idx[t-1]] += (x_bar - x_chosen[t-1])
    u = beta*X_jt[t] - gamma*Sigma[t]**2 + a[t] + epsilon_ijt[t] # love
        sameness
    V[t] = u
    chosen_idx[t] = np.argmax(V[t])
    x_chosen[t] = X_jt[t, chosen_idx[t]]

ll = -np.sum(V[np.arange(T), chosen_idx]-logsumexp(V,axis=1,keepdims=True))
prob = np.exp(V) / np.exp(V).sum(axis=1, keepdims=True)
prob_all[s] = prob
x_chosen_all[s] = x_chosen
x_bar_all[s] = x_bar
X_jt_all[s] = X_jt
V_all[s] = V
U_all[s] = V[np.arange(T), chosen_idx]
ll_all[s] = ll
return ConsResult(
    x_chosen_all.mean(axis=0), # shape (T,)
    x_bar_all.mean(axis=0),    # shape (T,)

```

```

        X_jt_all.mean(axis=0),          # shape (T, J)
        V_all.mean(axis=0),            # shape (T, J)
        prob_all.mean(axis=0),         # shape (T,J)
        U_all.mean(axis=0),            # shape (T,)
        ll_all.mean()
    )
# Utility Plots
colors = plt.cm.tab10(np.linspace(0,1,len(regimes)))

# Utility Plots
fig, ax = plt.subplots(figsize=(8, 4))
colors = plt.cm.tab10(np.linspace(0, 1, len(regimes)))

for (beta, gamma, label, ls), color in zip(regimes, colors):
    x_chosen, x_bar, X_jt, V, prob_all, U, ll = simulate_cons_markov(
        beta, gamma, kappa, sigma_gamma[1], T, J, X_bar, sigma_x, rng
    )
    ax.plot(np.arange(T), U, label=label, color=color, linewidth=1.5, linestyle=ls) #
    <-- ax.plot, not plt.plot

ax.set_xlabel("Period")
ax.set_ylabel("Utility")
ax.set_title("Consumer_Utility_by_Regime_(Markov)")
# put legend in top right
ax.legend(fontsize=7, loc="upper_right", bbox_to_anchor=(1,1))
plt.tight_layout()
plt.savefig("consumer_utility_by_regime_markov.pdf", bbox_inches='tight', format='pdf'
    )
plt.close()

for j in range(J):
    fig, ax = plt.subplots(figsize=(8, 4))
    for (beta, gamma, label, ls), color in zip(regimes, colors):
        x_chosen, x_bar, X_jt, V, prob, U, ll = simulate_cons_markov(
            beta, gamma, kappa, sigma_gamma[1], T, J, X_bar, sigma_x, rng
        )
        prob_smooth = np.convolve(prob[:, j], np.ones(5)/5, mode='same')
        ax.plot(np.arange(T), prob_smooth, label=label, color=color, linewidth=1.5,
            linestyle=ls)

    ax.set_title(f"Product_{j+1}_X={prod_space[j]:.1f})", fontsize=9)
    ax.set_xlabel("Period")
    ax.set_xlim(5, 95)
    ax.set_ylabel("Choice_Probability")
    ax.legend(fontsize=7, loc="upper_left", bbox_to_anchor=(1, 1))
    fig.suptitle("Choice_Probability_by_Product_and_Regime", fontsize=13, y=1.01)
    plt.tight_layout()

```

```

plt.savefig(f"choice_prob_product_{j+1}_markov.pdf", bbox_inches='tight', format='
pdf')
plt.close()

#=====#
# Firm Value Function w.r.t. CCP                                     #
#=====#

p = 2.0
mc = 1.0
beta_disc = 0.9

def firm_value_function(beta, gamma, kappa, beta_disc, mc, sigma_gamma, T, J, X_bar,
sigma_x, rng, p, S=1000):

    cons = simulate_cons(beta, gamma, kappa, sigma_gamma, T, J, X_bar, sigma_x, rng, S
    )

    prob = cons.prob_all
    X_bar_avg = cons.x_bar_all
    x_chosen = cons.x_chosen_all
    X_jt = cons.X_jt_all

    Vf = np.zeros((T+1, J))
    a_opt = np.zeros((T,J))
    V_diff = np.zeros((T,J))

    for t in range(T-1, -1, -1):
        eps_f = rng.standard_normal(size=J)
        for j in range(J):
            X_kt = X_jt[t].sum() - X_jt[t,j]
            s_jt = prob[t,j]
            for a_val in [0,1]:
                ad_cost = a_val
                pi = s_jt*(p-mc) - kappa*ad_cost**2
                EV_f = Vf[t+1, j]
                V_t = pi + beta_disc*EV_f
                if a_val == 0:
                    V_no_ad = V_t
                else:
                    V_ad = V_t

            V_diff[t,j] = V_ad - V_no_ad

            if V_ad >= V_no_ad:
                Vf[t,j] = V_ad
                a_opt[t,j] = 1
            else:

```

```

        Vf[t,j] = V_no_ad
        a_opt[t,j] = 0
    return Vf, a_opt, V_diff

from scipy.ndimage import uniform_filter1d

fig, axes = plt.subplots(1,2,figsize=(14, 4))
for (beta, gamma, label, ls), color in zip(regimes, colors):
    Vf, a, V_diff = firm_value_function(
        beta, gamma, kappa, beta_disc, mc, sigma_gamma, T, J, X_bar, sigma_x, rng, p,
        1000
    )
    smoothed = uniform_filter1d(a.mean(axis=1), size=10) # Smooth the average
        advertising choice over time
    axes[0].plot(np.arange(T), smoothed, label=label, color=color, linewidth=1.5,
        linestyle=ls) # <-- ax.plot, not plt.plot
    axes[1].plot(np.arange(T), V_diff.mean(axis=1), label=label, color=color,
        linewidth=1.5, linestyle=ls) # <-- ax.plot, not plt.plot

axes[0].set_xlabel("Period")
axes[0].set_ylabel("Pr(Advertise)")
axes[0].set_title("Smoothed_Ad_Probability_(Fixed)")
axes[0].axhline(0.5, color='k', linewidth=0.5, linestyle=':') # indifference line

axes[1].set_xlabel("Period")
axes[1].set_ylabel("V(ad)-V(no_ad)")
axes[1].set_title("Ad_Incentive_Over_Time_(Fixed)")
axes[1].axhline(0, color='k', linewidth=0.8, linestyle='--') # zero = indifference

handles, labels_leg = axes[1].get_legend_handles_labels()
fig.legend(handles, labels_leg, fontsize=7, loc='lower_center',
        bbox_to_anchor=(0.5, -0.15), ncol=3)

plt.tight_layout()
plt.savefig("ad_choice_both.pdf", bbox_inches='tight', format='pdf')
plt.close()

def simulate_firm_markov(beta, gamma, kappa, beta_disc, mc, sigma_gamma, T, J, X_bar,
    sigma_x, rng, p, S=1000):

    cons = simulate_cons_markov(beta, gamma, kappa, sigma_gamma, T, J, X_bar, sigma_x,
        rng, S)

    prob = cons.prob_all
    X_bar = cons.x_bar_all
    x_chosen = cons.x_chosen_all
    X_jt = cons.X_jt_all

```

```

Vf = np.zeros((T+1, J))
a_opt = np.zeros((T,J))
V_diff = np.zeros((T,J))

for t in range(T-1, -1, -1):
    eps_f = rng.standard_normal(size=J)
    for j in range(J):
        X_kt = X_jt[t].sum() - X_jt[t,j]
        s_jt = prob[t,j]
        markov_a = kappa*(X_bar[t] - x_chosen[t])**2
        for a_val in [0,markov_a]:
            ad_cost = a_val
            pi = s_jt*(p-mc) - ad_cost
            EV_f = Vf[t+1, j]
            V_t = pi + beta_disc*EV_f
            if a_val == 0:
                V_no_ad = V_t
            else:
                V_ad = V_t

        V_diff[t,j] = V_ad - V_no_ad

        if V_ad >= V_no_ad:
            Vf[t,j] = V_ad
            a_opt[t,j] = 1
        else:
            Vf[t,j] = V_no_ad
            a_opt[t,j] = 0
    return Vf, a_opt, V_diff

fig, axes = plt.subplots(1,2,figsize=(14, 4))
for (beta, gamma, label, ls), color in zip(regimes, colors):
    Vf_m, a_m, V_diff_m = simulate_firm_markov(
        beta, gamma, kappa, beta_disc, mc, sigma_gamma, T, J, X_bar, sigma_x, rng, p,
        1000
    )
    smoothed_m = uniform_filter1d(a_m.mean(axis=1), size=10) # Smooth the average
                    advertising choice over time
    axes[0].plot(np.arange(T), smoothed_m, label=label, color=color, linewidth=1.5,
                 linestyle=ls)
    axes[1].plot(np.arange(T), V_diff_m.mean(axis=1), label=label, color=color,
                 linewidth=1.5, linestyle=ls)

axes[0].set_xlabel("Period")
axes[0].set_ylabel("Pr(Advertise)")
axes[0].set_title("Smoothed_Ad_Probability_(Markov)")
axes[0].axhline(0.5, color='k', linewidth=0.5, linestyle=':') # indifference line

```

```

axes[1].set_xlabel("Period")
axes[1].set_ylabel("V(ad)-V(no_ad)")
axes[1].set_title("Ad_Incentive_Over_Time_(Markov)")
axes[1].axhline(0, color='k', linewidth=0.8, linestyle='--') # zero = indifference

handles, labels_leg = axes[1].get_legend_handles_labels()
fig.legend(handles, labels_leg, fontsize=7, loc='lower_center',
           bbox_to_anchor=(0.5, -0.15), ncol=3)

plt.tight_layout()
plt.savefig("ad_choice_markov_both.pdf", bbox_inches='tight', format='pdf')
plt.close()

```